HEAT TRANSFER PIPELINE

Complete Line-by-Line Explanation

2. MESH SECTION - GMSH Script

Overview:
GMSH (Gmsh mesh generator) takes the CAD geometry in STEP format and creates a computational mesh. The mesh divides the 3D space into small elements (tetrahedra or hexahedra) for the FEM solver to use.

Factory Setup and Import

```
SetFactory("OpenCASCADE");
Merge "Gemini 5.step";
```

SetFactory("OpenCASCADE");
Sets GMSH to use the OpenCASCADE kernel. This kernel understands STEP files and provides robust geometry handling. Alternative kernels like Geo are simpler but less capable.

Merge "Gemini 5.step";
Imports the CAD geometry from the STEP file created by FreeCAD. The geometry is now available in GMSH for meshing.

Physical Groups Definition

```
Physical Surface(78) = {20, 16, 21, 17, 19, 25, 23, 24, 22, 18, 15, 28, 14, 6, 13,
11, 10, 12, 3, 27, 7, 5, 4, 9, 8, 2, 26, 1};
Physical Volume(79) = {1};
```

Physical Surface(78) = {...}:
Defines a physical group named "78" containing all surface faces. These numbers (20, 16, 21, etc.) are the internal GMSH surface IDs from the imported geometry. Physical groups are used to assign boundary conditions in the solver.

Physical Volume(79) = {1};
Defines a physical group containing the 3D volume (interior) of the solid. The "1" refers to the first (and usually only) volume in the geometry.

Heal & Merge Volumes (Critical Step)

```
all_vols[] = Volume { : };
BooleanFragments{ Volume{all_vols[]}; }{ }
```

all_vols[] = Volume { : };
Selects all volumes in the geometry. The colon syntax means "all". This creates an array of volume IDs.

BooleanFragments{ Volume{all_vols[]}; }{ }
Performs a Boolean fragment operation on all volumes. Even though the FreeCAD geometry was already

fused, GMSH sometimes sees it as a "hollow shell". This command forces GMSH to treat it as a solid piece and fill the interior with tetrahedra.

Mesh Size Settings

```
Mesh.MeshSizeMin = 1;
Mesh.MeshSizeMax = 2;
```

Mesh.MeshSizeMin = 1;

Sets the minimum element edge length to 1mm. GMSH won't create elements smaller than this.

Mesh.MeshSizeMax = 2;

Sets the maximum element edge length to 2mm. Most elements will be around this size. Smaller values create finer meshes (more elements, slower solving) but more accurate results. Larger values are faster but coarser.

Transfinite & Recombination Loop

```
all_surfs[] = Surface { : };

For i In {0 : #all_surfs[]-1}
    Transfinite Surface {all_surfs[i]};
    Recombine Surface {all_surfs[i]};
EndFor
```

all_surfs[] = Surface { : };

Selects all surfaces in the geometry and stores their IDs in an array.

For i In {0 : #all_surfs[]-1}

Loop syntax in GMSH. i ranges from 0 to the number of surfaces minus 1. #all_surfs[] gets the array length.

Transfinite Surface {all_surfs[i]};

Applies transfinite meshing to surface i. This creates a structured grid of quadrilaterals instead of triangles, which often gives better solver results and faster convergence.

Recombine Surface {all_surfs[i]};

Recombines triangular elements into quadrilaterals on surface i. After transfinite meshing, this ensures we get quad-dominant meshes on the boundary.

Volume Meshing Algorithms

```
Mesh.Algorithm = 6;
Mesh.Algorithm3D = 1;
Mesh.RecombineAll = 1;
```

Mesh.Algorithm = 6;

Sets the 2D meshing algorithm to 6 (Frontal-Delaunay). This is good for creating quad-dominant surface meshes. Algorithm values: 1=MeshAdapt, 2=Automatic, 3=Initial, 5=Delaunay, 6=Frontal-Delaunay, 7=BAMG, 8=Frontal-Delaunay for Quads.

Mesh.Algorithm3D = 1;
Sets the 3D meshing algorithm to 1 (Delaunay). Delaunay is fast and creates good tetrahedra. Alternative: 10=HXT (newer, parallel).

Mesh.RecombineAll = 1;
Allows some flexibility: if quad recombination fails, keep some triangles instead of forcing all quads. Value 1 enables this fallback.

Physical Volume Definition for Solver

```
Physical Volume("Solid_Impeller", 80) = {all_vols[]};
```

Physical Volume("Solid_Impeller", 80) = {all_vols[]}:
Defines a physical volume group with name "Solid_Impeller" and ID 80, containing all volumes. The solver uses this physical group to know which region is the solid material to solve on.

3. SOLVER SECTION - Elmer FEM

Overview:
Elmer is an open-source Finite Element Method (FEM) solver. It solves the heat equation with appropriate boundary and initial conditions. The .sif (Solver Input File) configuration file specifies all solver parameters.

Header Section

```
Header
  CHECK KEYWORDS Warn
  Mesh DB "." "."
  Include Path ""
  Results Directory ""
End
```

CHECK KEYWORDS Warn:
Tells Elmer to warn (but not fail) if unknown keywords are found. Useful for debugging typos.

Mesh DB "." ".":
Specifies where mesh files are located. Both dots mean current directory. Format: "Mesh directory" "Mesh name".

Include Path "":
Directory for included files. Empty means no additional include paths.

Results Directory "":
Where to save output files. Empty means current directory.

Simulation Settings

```
Simulation
  Max Output Level = 5
  Coordinate System = Cartesian
  Coordinate Mapping(3) = 1 2 3
  Simulation Type = Transient
  Steady State Max Iterations = 1
  Output Intervals(1) = 1
  Timestep intervals(1) = 500
  Timestep Sizes(1) = 0.1
  Timestepping Method = BDF
  BDF Order = 2
  Solver Input File = case.sif
  Post File = case.vtu
End
```

Max Output Level = 5:
Controls verbosity. 0=quiet, 5=normal, 10=very verbose. Level 5 prints key solver info and convergence details.

Coordinate System = Cartesian:
Uses Cartesian coordinates (x, y, z) instead of cylindrical or spherical.

Coordinate Mapping(3) = 1 2 3:
Maps the 3 coordinate axes in order: 1=X, 2=Y, 3=Z. Standard ordering.

Simulation Type = Transient:
Transient means time-dependent. The temperature evolves over time. Alternative: "Steady State" for time-independent.

Steady State Max Iterations = 1:
For transient simulations, Elmer uses pseudo-time steps. This limits iterations per timestep to 1.

Output Intervals(1) = 1:
Output results every 1 timestep. Value 2 would output every 2 steps, etc.

Timestep intervals(1) = 500:
Number of time steps to simulate. Array format allows multiple intervals with different timestep sizes.

Timestep Sizes(1) = 0.1:
Duration of each timestep in seconds. 0.1 second per step × 500 steps = 50 seconds total simulation time.

Timestepping Method = BDF:
BDF (Backward Differentiation Formula) is a high-order time integration method. More stable for stiff equations than Euler method.

BDF Order = 2:
Uses 2nd-order BDF. Higher order = more accurate but requires more history. Order 1 is similar to backward Euler.

Solver Input File = case.sif and Post File = case.vtu:
case.sif is the main solver file. case.vtu is the output format (VTK unstructured grid), readable by ParaView.

Physical Constants

```
Constants
  Gravity(4) = 0 -1 0 9.82
  Stefan Boltzmann = 5.670374419e-08
  Permittivity of Vacuum = 8.85418781e-12
  Permeability of Vacuum = 1.25663706e-6
  Boltzmann Constant = 1.380649e-23
  Unit Charge = 1.6021766e-19
End
```

Gravity(4) = 0 -1 0 9.82:
Defines gravity vector. Values are [Gx, Gy, Gz, |G|]. (0, -1, 0) means gravity points in -Y direction with magnitude 9.82 m/s^2. Not used in pure heat transfer but available for coupled problems.

Stefan Boltzmann = 5.670374419e-08:
Stefan-Boltzmann constant used for radiation heat transfer: $\sigma = 5.67\times10^{-8}$ $W/(m^2\cdot K^4)$. Units: SI.

Permittivity of Vacuum = 8.85418781e-12:
Vacuum permittivity $\varepsilon_0 = 8.854\times10^{-12}$ F/m. Relevant for electromagnetic problems, not heat transfer.

Permeability of Vacuum = 1.25663706e-6:
Vacuum permeability $\mu_0 = 1.257 \times 10^{-6}$ H/m. Relevant for electromagnetic problems.

Boltzmann Constant = 1.380649e-23 and Unit Charge = 1.6021766e-19:
Fundamental physical constants. Not used in basic heat transfer.

Body Definition

```
Body 1
  Target Bodies(1) = 1
  Name = "Body 1"
  Equation = 1
  Material = 1
  Initial condition = 1
End
```

Target Bodies(1) = 1:
Links this body block to mesh body 1 (the solid impeller). If mesh had multiple bodies, use different numbers.

Name = "Body 1":
Human-readable name for the body.

Equation = 1:
Links to Equation block 1 (defines what physics to solve on this body).

Material = 1:
Links to Material block 1 (specifies material properties like thermal conductivity).

Initial condition = 1:
Links to Initial Condition block 1 (sets starting temperature).

Solver Configuration (Heat Equation)

```
Solver 1
  Equation = Heat Equation
  Variable = Temperature
  Procedure = "HeatSolve" "HeatSolver"
  Exec Solver = Always
```

Equation = Heat Equation:
Specifies which physics equation to solve. Elmer automatically uses the appropriate solver module for heat equations.

Variable = Temperature:
The unknown variable being solved for. The heat equation solves for T (temperature) at each mesh node.

Procedure = "HeatSolve" "HeatSolver":
Specifies the solver library and subroutine. "HeatSolve" is the library, "HeatSolver" is the entry point function.

Exec Solver = Always:
Execute this solver in every timestep. Alternative: "Never" to skip, or "After Timestep 5" for conditional execution.

Stabilization & Convergence

```
Stabilize = True
Optimize Bandwidth = True
Steady State Convergence Tolerance = 1.0e-5
Nonlinear System Convergence Tolerance = 1.0e-7
Nonlinear System Max Iterations = 20
Nonlinear System Newton After Iterations = 3
Nonlinear System Newton After Tolerance = 1.0e-3
Nonlinear System Relaxation Factor = 1
```

Stabilize = True:
Enables stabilization (like SUPG) to prevent numerical oscillations in advection-dominated flows. For pure heat conduction, less critical but still helps.

Optimize Bandwidth = True:
Reorders equations to minimize the bandwidth of the stiffness matrix. Reduces memory and improves solver speed.

Steady State Convergence Tolerance = 1.0e-5:
For each timestep (treated as pseudo steady-state), nonlinear iterations stop when residual < 1.0×10^{-5}.

Nonlinear System Convergence Tolerance = 1.0e-7:
Stricter tolerance for the nonlinear system. Iterations stop when change in temperature < 1.0×10^{-7} K.

Nonlinear System Max Iterations = 20:
Maximum of 20 nonlinear iterations per timestep. Stops earlier if convergence tolerance is met.

Nonlinear System Newton After Iterations = 3:
Use simple fixed-point iterations for first 3 iterations, then switch to Newton's method for faster convergence.

Nonlinear System Newton After Tolerance = 1.0e-3:
Switch to Newton method once residual drops below 1.0×10^{-3}.

Nonlinear System Relaxation Factor = 1:
Relaxation factor for nonlinear iterations. Value 1.0 means no relaxation; less than 1 dampens oscillations.

Linear System Solver Settings

```
Linear System Solver = Iterative
Linear System Iterative Method = BiCGStab
Linear System Max Iterations = 500
Linear System Convergence Tolerance = 1.0e-10
BiCGstabl polynomial degree = 2
Linear System Preconditioning = ILU0
Linear System ILUT Tolerance = 1.0e-3
Linear System Abort Not Converged = False
```

```
  Linear System Residual Output = 10
  Linear System Precondition Recompute = 1
```

Linear System Solver = Iterative:
Uses iterative solver (like conjugate gradient) instead of direct solver (like LU decomposition). Iterative is faster for large systems.

Linear System Iterative Method = BiCGStab:
BiCGStab (BiConjugate Gradient Stabilized) is a Krylov subspace method. Works well for general non-symmetric matrices. Alternative: "CG" for symmetric matrices.

Linear System Max Iterations = 500:
Maximum iterations to solve the linear system per nonlinear step. Stops earlier if convergence reached.

Linear System Convergence Tolerance = 1.0e-10:
Very tight tolerance (1.0×10^{-10}). Solves the linear system accurately. Typical range: 1e-8 to 1e-12.

BiCGstabl polynomial degree = 2:
Uses BiCGSTABL variant with polynomial degree 2. Higher degree = slower convergence per iteration but better stability.

Linear System Preconditioning = ILU0:
Preconditioning strategy. ILU0 (Incomplete LU decomposition level 0) speeds up convergence. Alternative: "ILUT" with tolerance.

Linear System ILUT Tolerance = 1.0e-3:
Drop tolerance for ILUT preconditioner. Smaller values = denser preconditioner = more memory but better convergence.

Linear System Abort Not Converged = False:
If False, continue anyway even if linear system doesn't converge. If True, stop simulation on convergence failure.

Linear System Residual Output = 10:
Print residual every 10 iterations. Helps monitor solver progress. 0 = no output.

Linear System Precondition Recompute = 1:
Recompute preconditioner every timestep. Value 1 = recompute always, larger values = recompute every N timesteps.

Equation Block

```
Equation 1
  Name = "Equation 1"
  Active Solvers(1) = 1
End
```

Name = "Equation 1":
Human-readable equation name.

Active Solvers(1) = 1:
Links this equation to Solver block 1. Multiple solvers can be active (coupled problems).

Material Properties

```
Material 1
  Name = "Aluminium (generic)"
  Poisson ratio = 0.35
  Density = 2700.0
  Sound speed = 5000.0
  Heat Conductivity = 237.0
  Youngs modulus = 70.0e9
  Heat expansion Coefficient = 23.1e-6
  Heat Capacity = 897.0
End
```

Name = "Aluminium (generic)":
Material name. Identifies the material type.

Density = 2700.0:
Mass density in kg/m^3. For heat transfer, used to compute heat capacity per volume.

Heat Conductivity = 237.0:
Thermal conductivity in W/(m·K). How fast heat diffuses through the material. High value (237) means excellent conductor (aluminum).

Heat Capacity = 897.0:
Specific heat capacity in J/(kg·K). Energy needed to raise 1 kg by 1 K. Controls how temperature changes with heat added.

Poisson ratio = 0.35, Youngs modulus = 70.0e9:
Mechanical properties (not used in pure heat transfer). Included for reference; useful if solving coupled thermo-mechanical problems.

Sound speed = 5000.0:
Speed of sound in the material (m/s). Not used in heat transfer; included for completeness.

Heat expansion Coefficient = 23.1e-6:
Volumetric thermal expansion coefficient in K^{-1}. Controls how dimensions change with temperature.

Initial Condition

```
Initial Condition 1
  Name = "InitialCondition 1"
  Temperature = 1000
End
```

Temperature = 1000:
Starting temperature everywhere in the domain: 1000 K (about 727°C). At time t=0, all points are at this temperature.

Boundary Condition

```
Boundary Condition 1
  Target Boundaries(1) = 1
  Name = "BoundaryCondition 1"
  Heat Transfer Coefficient = 800
  External Temperature = 300
End
```

Target Boundaries(1) = 1:

Applies this boundary condition to boundary group 1 (all outer surfaces from Physical Surface 78).

Heat Transfer Coefficient = 800:

Convection coefficient in $W/(m^2 \cdot K)$. Describes cooling strength. Formula: q_out = h × (T_surface - T_external). Higher h = stronger cooling.

External Temperature = 300:

Ambient temperature in K (about 27°C, room temperature). The surrounding air is at this temperature.

4. POST PROCESSING - ParaView Visualization

Overview:
After the solver finishes, the output file (case.vtu) contains temperature values at all mesh nodes. ParaView is used to visualize this data, creating colored contour plots showing temperature distribution.

ParaView Visualization Setup

```
Temperature Range: 1000K to 950K
Color Scale: Blue (cold) → Red (hot)
```

Temperature Range: 1000K to 950K:
Sets the display scale from 1000 K (initial condition) to 950 K (final condition after 50 seconds of cooling). The impeller cools from 1000 K to approximately 950 K as it loses heat via convection.

Color Scale:
Temperature values are shown as colors. Typically: blue for cold (950K), red for hot (1000K), with green/yellow in between. This allows quick visual identification of hot and cold regions.

Complete Pipeline Summary

The heat transfer pipeline consists of four tightly integrated stages:

STAGE 1 - CAD (FreeCAD Python):
Creates a parametric 3D impeller model with spiral blades using logarithmic geometry. The script reads dimensions from a CSV file, generates smooth curves using B-splines, assembles multiple blades through rotation, and removes duplicate surfaces for clean geometry suitable for meshing.

STAGE 2 - MESH (GMSH):
Converts the CAD geometry (STEP format) into a computational mesh. It heals the geometry, sets element sizes (1-2mm), applies transfinite structured meshing on surfaces, and uses Delaunay algorithm for 3D volume meshing. The result is a tetrahedral mesh with ~10,000-100,000 elements.

STAGE 3 - SOLVER (Elmer FEM):
Solves the unsteady (transient) heat equation: $\rho C_p(\partial T/\partial t) = \nabla\cdot(k\nabla T) + q$, where ρ is density, C_p is specific heat, T is temperature, k is thermal conductivity, and q is heat source/sink. Boundary condition: convective cooling at the surface. Time integrator: 2nd-order BDF, converges over 500 timesteps (50 seconds).

STAGE 4 - POST PROCESSING (ParaView):
Visualizes temperature evolution. Shows color maps (blue=cold, red=hot) and temperature contours. Helps identify hot spots, verify cooling rates, and validate solver results against expectations.

Physics Insight:
Initial condition: 1000 K uniform temperature everywhere
Boundary condition: Convective cooling with h=800 $W/(m^2\cdot K)$, external temperature 300 K
Heat transfer coefficient equation: q = h(T_surface - T_ambient)
Material: Aluminum with conductivity 237 $W/(m\cdot K)$, density 2700 kg/m^3, specific heat 897 $J/(kg\cdot K)$
Expected result: Temperature decays exponentially toward ambient as internal heat diffuses outward and is removed at the boundaries.

Key Parameters and Their Effects:
- Mesh size (1-2mm): Finer mesh → more accurate but slower
- Timestep (0.1s): Smaller → more accurate but more steps needed
- BDF Order (2): Higher order → more accurate but more history required
- Convection coefficient (800): Higher → faster cooling
- Thermal conductivity (237): Higher → faster internal heat diffusion